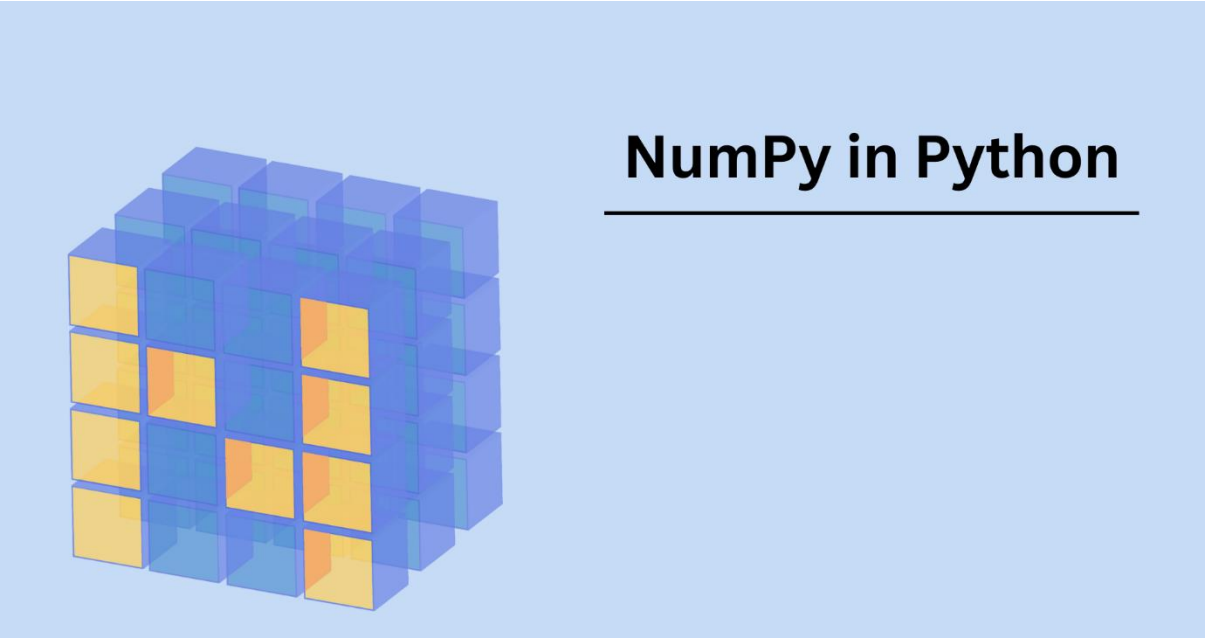
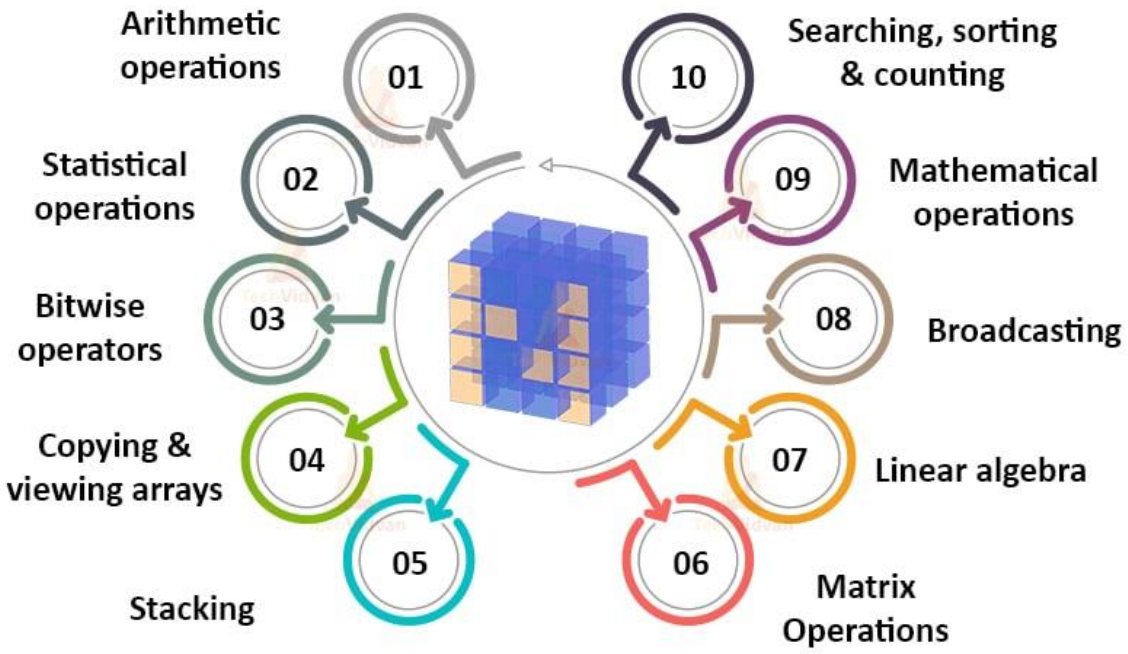
Comprehensive Guide to NumPy Arrays



Introduction

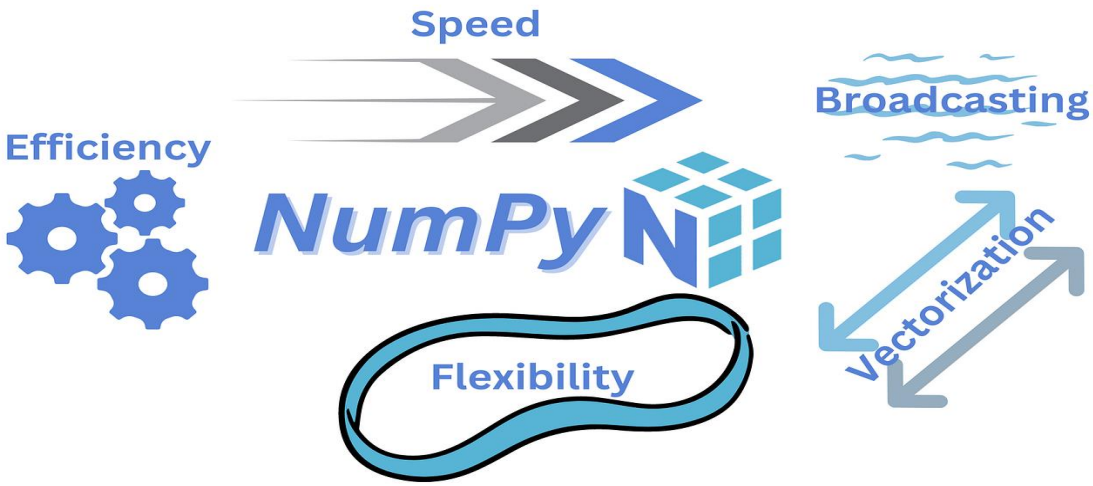
- NumPy stands for Numerical Python.
- It provides efficient handling of large multi-dimensional arrays and matrices.
- Supports mathematical, logical, and statistical operations.

Uses of NumPy



What is NumPy?

- A Python library for numerical computations.
- Supports arrays, matrices, and high-level mathematical functions.
- Optimized in C for performance.



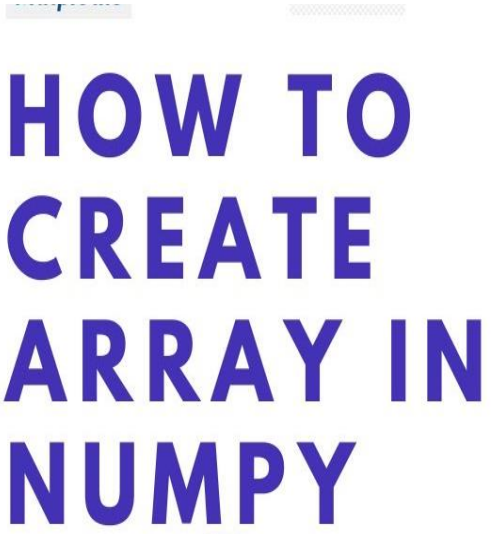
Why Not Use Python Lists?

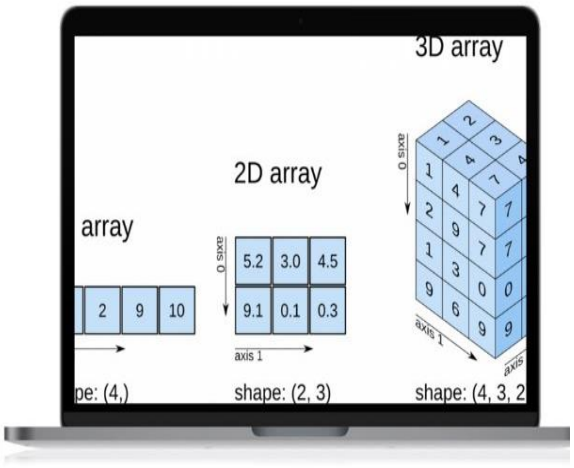
- Lists are slower and consume more memory.
- Do not support vectorized operations.
- NumPy arrays are faster and more efficient for large data.

List vs NumPy Array

Python Lists	Python Arrays
Collects items of multiple data types	Collects items of same data type
Uses more memory	Uses less memory
Flexible, Easy to modify	Not fexible, Difficult to modify
No need for a specific loop to access the components	Needs a loop to access the components of array
Good for shorter sequence of data	Good for longer sequence of data

Ways to Create NumPy Arrays





np.array**(list/tuple)**

ages = [23,45,67,34]

ar1 = np.array(ages)

type(ar1)

numpy.ndarray

#from tuple

ages = (23,45,67,34)

ar2=np.array(ages)

type(ar2)

np.arrange()

-To generate sequence of array values

```
np.arrange(1,20,2)
```

Output:

```
array([1,3,5,7,9,11,13,15,17,19])
```

np.random.randit() # To generate random numbers in the given limits.

```
np.random.randint(10,100,20)
```

Output:

```
array([63, 74, 49, 83, 53, 42, 37, 81, 94, 19, 77, 52, 45, 38, 43, 35, 87,
       57, 55, 50]
```

np.linspace() #To generate linearly spaced (equally spaced values)

```
np.linspace(10,30,5)
array([10., 15., 20., 25., 30.])
```

np.zeros ()

To generate all the values are zeros

```
np.zeros((2,3))
array([[0., 0., 0.],
       [0., 0., 0.]])
```

np.ones() To generate an array with all 1's

```
np.ones((4,5))
```

Output:

```
array([[1., 1., 1., 1., 1.],
       [1., 1., 1., 1., 1.],
       [1., 1., 1., 1., 1.],
       [1., 1., 1., 1., 1.]])
```

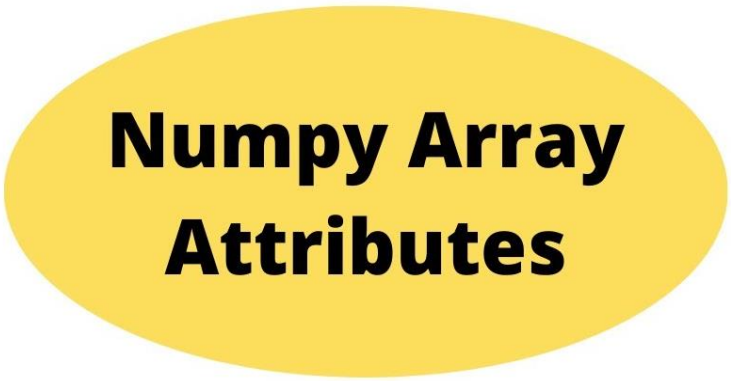
np.full() # To generate an array with same value

```
np.full((2,3),8)
```

Output:

```
array([[8, 8, 8],
       [8, 8, 8]])
```

Attributes of an Array



arr.shape # No of rows and columns

arr.ndim # no of dimensional

arr.dtype # Data Types of values in the array

arr.itemsize #The memory occupied by single value in the array (in bytes)

arr. nbytes # Total memory occupied bby array (in Bytes)

arr.size # No of values present in the array

Statistical Measures

Measure of Central Tendancy

```
np.random.seed(435)
arr = np.random.randint(1,100,40)
# Mean
np.mean(arr)
```

output:

```
np.float64(49.0)
```

```
#median
np.median(arr)
```

Output:

```
np.float64(51.0)
```

Measure of dispersion

```
np.max(arr)
```

Output:

```
np.int32(98)
```

```
np.min(arr)
```

Output:

```
np.int32(2)
```

```
np.var(arr,ddof=0)
```

Output:

```
np.float64(995.25)
```

```
range_ = np.max(arr) - np.min(arr)
```

```
np.std(arr,ddof=0)
```

Output:

```
np.float64(31.547583108694713)
```

```
q3 = np.percentile(arr,75)
```

```
q1 = np.percentile(arr,25)
```

```
np.quantile(arr,0.75)
```

Output:

np.float64(72.75)

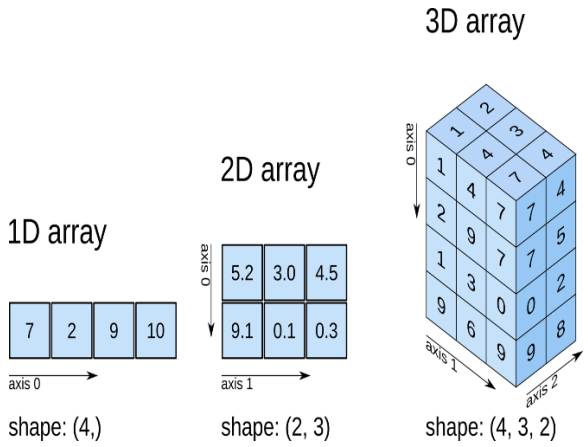
iqr = q3-q1

iqr

Output:

np.float64(55.0)

Multi Dimensional Arrays



View v/s Copy

import numpy as np

arr = np.arange(1,10)

- view - original array will be impacted
- copy - original array willl not be impacted

```
arr_view = arr.view()
arr_copy = arr.copy()
arr_copy
```

Output:

```
array([1, 2, 3, 4, 5, 6, 7, 8, 9])
arr_view[2] = 1000
arr
```

Output:

```
array([   1,    2, 1000,  4,    5,    6,    7,    8,    9])
arr_copy[4]=3000
arr_copy
```

Output:

```
array([   1,    2,    3,    4, 3000,    6,    7,    8,    9])
```

While creating

```
a = np.array([1,2,3,4,5],ndmin = 5)
a
```

Output:

```
array([[[[[1, 2, 3, 4, 5]]]]])
```

After Creation

```
arr = np.random.randint(10,50,12)

arr
```

Output:

```
array([38, 21, 40, 20, 30, 47, 21, 28, 46, 26, 10, 46], dtype=int32)
```

```
np.expand_dims(arr,axis = 1)
```

Output:

```
array([[38],
       [21],
       [40],
       [20],
       [30],
       [47],
       [21],
       [28],
       [46],
       [26],
       [10],
       [46]], dtype=int32)
```

Reshape

```
np.reshape(arr,(4,3))
```

Output:

```
array([[38, 21, 40],
       [20, 30, 47],
       [21, 28, 46],
       [26, 10, 46]], dtype=int32)
```

```
np.resize()
arr = np.random.randint(10,50,12)
arr
```

Output:

```
array([13, 23, 32, 41, 30, 25, 28, 20, 11, 36, 44, 44], dtype=int32)
np.resize(arr,(4,3))
```

Output:

```
array([[13, 23, 32],
       [41, 30, 25],
```

[28, 20, 11],

[36, 44, 44]], dtype=int32)

Flatten vs Ravel in Numpy

Flatten()

Returns as a copy of the array in 1D.

Any changes made the to the flattened array do not effect the original .

Slower (creates new memory)

Ravel()

Returns a view whenever possible of the array in 1D

Changes in ravelled array may affect the original

Faster (no memory allocation

Indexing, Slicing, and Boolean Indexing

Indexing

Indexing means accessing elements of a NumPy array using their position.

- NumPy uses **zero-based indexing** → first element = index 0.
- For **2D arrays**, we need both row and column indices.

Example (1D Array)

```
import numpy as np


arr = np.array([10, 20, 30, 40, 50])
print(arr[0])
# 10 (first element)
print(arr[2])
# 30 (third element)
print(arr[-1])
# 50 (last element)
```

Example (2D Array)

```
arr2d = np.array([[1, 2, 3], [4, 5, 6]])
print(arr2d[0, 1])


 # 2  (row 0, col 1)
print(arr2d[1, 2])   # 6  (row 1, col 2)
```

Slicing

Slicing is used to extract **subsets of arrays**.
Syntax → [start : stop : step]

- start → starting index (default = 0)
- stop → ending index (not included)
- step → gap between elements (default = 1)

Example (1D Array)

```
arr = np.array([10, 20, 30, 40, 50, 60, 70])

print(arr[1:5])

# [20 30 40 50]

print(arr[:4])

# [10 20 30 40]

print(arr[::2])

# [10 30 50 70]
```

Example (2D Array)

```
arr2d = np.array([[1, 2, 3],
                  [4, 5, 6],
                  [7, 8, 9]])

print(arr2d[0:2, 1:3])

# [[2 3]

#  [5 6]]
```

Boolean Indexing

Boolean indexing allows selecting elements using **conditions**.
It creates a **True/False mask** which NumPy applies to the array.

Example (1D Array)

```
arr = np.array([10, 20, 30, 40, 50])

print(arr > 25)       # [False False  True  True  True]

print(arr[arr > 25])  # [30 40 50]
```

Multiple Conditions

```
print(arr[(arr > 15) & (arr < 45)])

 # [20 30 40]

print(arr[(arr % 20 == 0)])

 # [20 40]
```

Mathematical and Trigonometric Functions in NumPy

Mathematical Functions in NumPy

NumPy provides a wide variety of mathematical functions that make calculations easy and efficient. These functions work element-wise on arrays.

(a) Basic Arithmetic Functions

- **np.add(x, y)** → Adds two arrays element-wise.
- **np.subtract(x, y)** → Subtracts second array from first.
- **np.multiply(x, y)** → Multiplies arrays element-wise.
- **np.divide(x, y)** → Divides first array by second element-wise.

Example:

```
import numpy as np

a = np.array([10, 20, 30])

b = np.array([2, 5, 10])


print("Add:", np.add(a, b))

 # [12 25 40]

print("Subtract:", np.subtract(a, b))


# [ 8 15 20]

print("Multiply:", np.multiply(a, b))

# [ 20 100 300]
```

```
print("Divide:", np.divide(a, b))
 # [5. 4. 3.]
```

(b) Power and Exponential Functions

- **np.power(x, y)** → Raises array elements to the power y.
- **np.sqrt(x)** → Square root of elements.
- **np.exp(x)** → Exponential (e^x) of each element.
- **np.log(x)** → Natural logarithm (base e).
- **np.log10(x)** → Base-10 logarithm.

Example:

```
x = np.array([1, 2, 3, 4])
print("Power:", np.power(x, 3))
# [ 1  8 27 64]
print("Square Root:", np.sqrt(x))
 # [1. 1.41 1.73 2.]
print("Exponential:", np.exp(x))
 # [ 2.71  7.38 20.08 54.59]
print("Log:", np.log(x))
 # [0. 0.69 1.09 1.39]
print("Log10:", np.log10(x))
 # [0. 0.30 0.47 0.60]
```

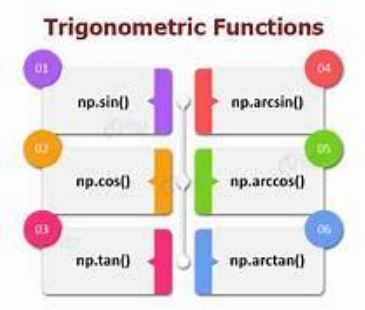
(c) Rounding and Related Functions

- **np.round(x)** → Rounds to nearest integer.
- **np.floor(x)** → Rounds down.
- **np.ceil(x)** → Rounds up.
- **np.trunc(x)** → Removes decimal part (truncate).

Example:

```
y = np.array([2.7, -3.4, 5.9])
print("Round:", np.round(y))
# [ 3. -3.  6.]
print("Floor:", np.floor(y))

# [ 2. -4.  5.]
print("Ceil:", np.ceil(y))
# [ 3. -3.  6.]
print("Trunc:", np.trunc(y))   # [ 2. -3.  5.]
```

Trigonometric Functions in NumPy



NumPy supports a wide range of trigonometric functions (in radians).

(a) Basic Trigonometric Functions

- **np.sin(x)** → Sine values.
- **np.cos(x)** → Cosine values.
- **np.tan(x)** → Tangent values.

Example:

```
angles = np.array([0, np.pi/6, np.pi/4, np.pi/3, np.pi/2])
print("Sine:", np.sin(angles))
print("Cosine:", np.cos(angles))
print("Tangent:", np.tan(angles))
```

Output:

```
Sine: [0.    0.5   0.707 0.866 1.  ]
Cosine: [1.    0.866 0.707 0.5   0.  ]
Tangent: [0.    0.577 1.    1.732 inf ]
```

(b) Inverse Trigonometric Functions

- **np.arcsin(x)** → Inverse sine.
- **np.arccos(x)** → Inverse cosine.
- **np.arctan(x)** → Inverse tangent.

Example:

```
values = np.array([0, 0.5, 1])
print("arcsin:", np.arcsin(values))
 # in radians
print("arccos:", np.arccos(values))
print("arctan:", np.arctan(values))
```

(c) Angle Conversions

- **np.deg2rad(x)** → Converts degrees to radians.
- **np.rad2deg(x)** → Converts radians to degrees.

Example:

```
deg = np.array([0, 30, 45, 90])
rad = np.deg2rad(deg)
print("Degrees to Radians:", rad)
print("Radians to Degrees:", np.rad2deg(rad))
```

(d) Hyperbolic Functions

- **np.sinh(x)** → Hyperbolic sine.
- **np.cosh(x)** → Hyperbolic cosine.
- **np.tanh(x)** → Hyperbolic tangent.

Example:

```
z = np.array([0, 1, 2])
print("sinh:", np.sinh(z))
print("cosh:", np.cosh(z))
print("tanh:", np.tanh(z))
```

Array Manipulation in NumPy

Array manipulation in NumPy refers to **reshaping, joining, splitting, inserting, deleting, and changing the structure** of arrays. These operations are extremely useful in **data processing, machine learning, and numerical analysis**

```
arr = np.array([[1,2,3],[4,5,6],[7,8,9]])
arr
```

Output:

```
array([[1, 2, 3],
       [4, 5, 6],
       [7, 8, 9]])
np.fliplr(arr)      # change left to right
```

Output:

```
array([[3, 2, 1],
       [6, 5, 4],
       [9, 8, 7]])

np.flipud(arr)  # change up to down
```

Output:

```
array([[7, 8, 9],
       [4, 5, 6],
       [1, 2, 3]])
np.flip(arr)
```

Output:

```
array([[9, 8, 7],
       [6, 5, 4],
       [3, 2, 1]])
np.rot90(arr)
```

Output:

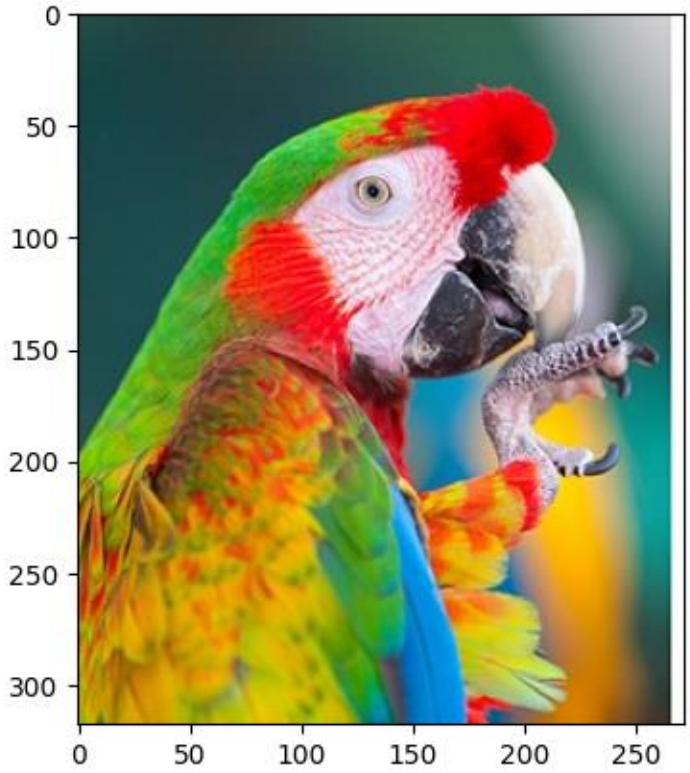
```
array([[3, 6, 9],
       [2, 5, 8],
       [1, 4, 7]])
```

Example

```
!pip install matplotlib
import matplotlib.pyplot as plt
image = plt.imread(r"C:\Users\ghema\Downloads\image.jpg")
```

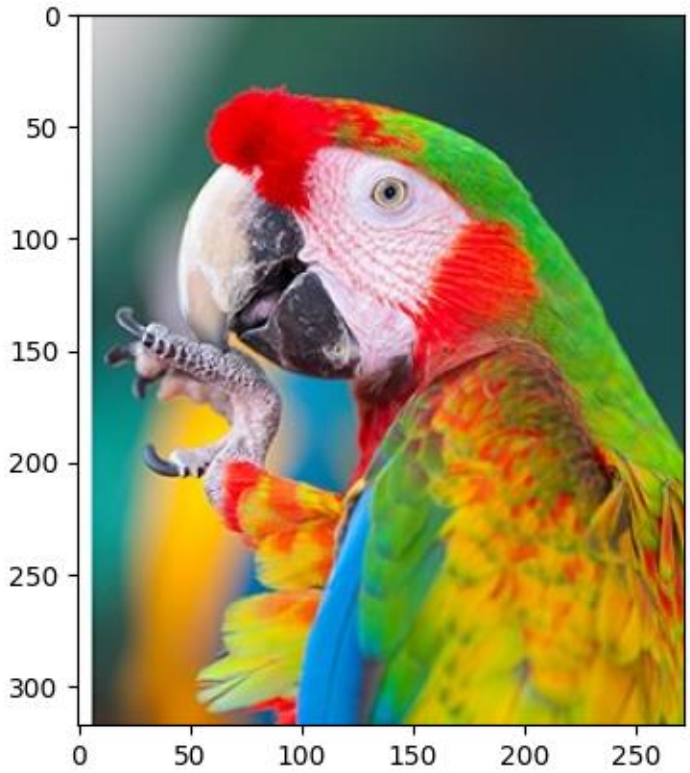
```
plt.imshow(image)
```

```
<matplotlib.image.AxesImage at 0x1c570da92b0>
```

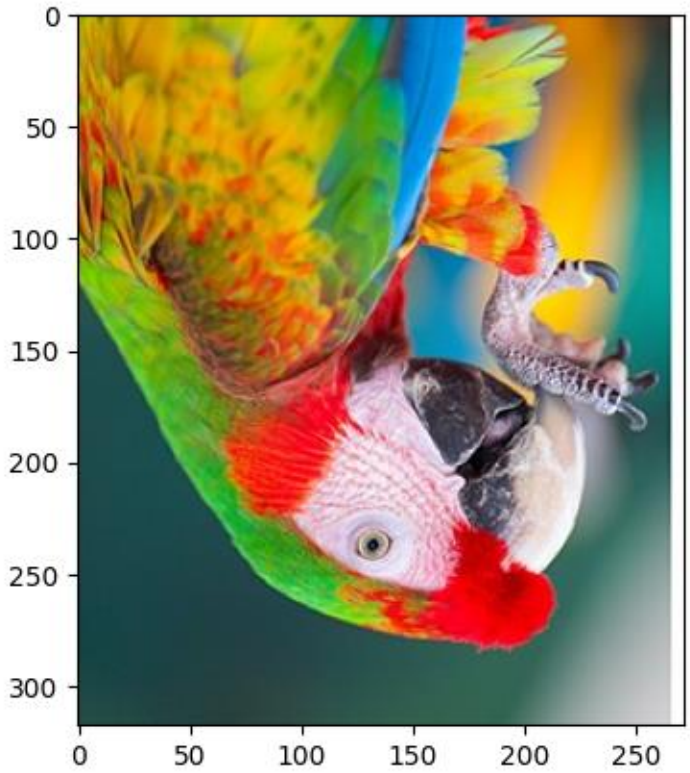



```
flipped_image  = np.fliplr(image)
plt.imshow(flipped_image)
```

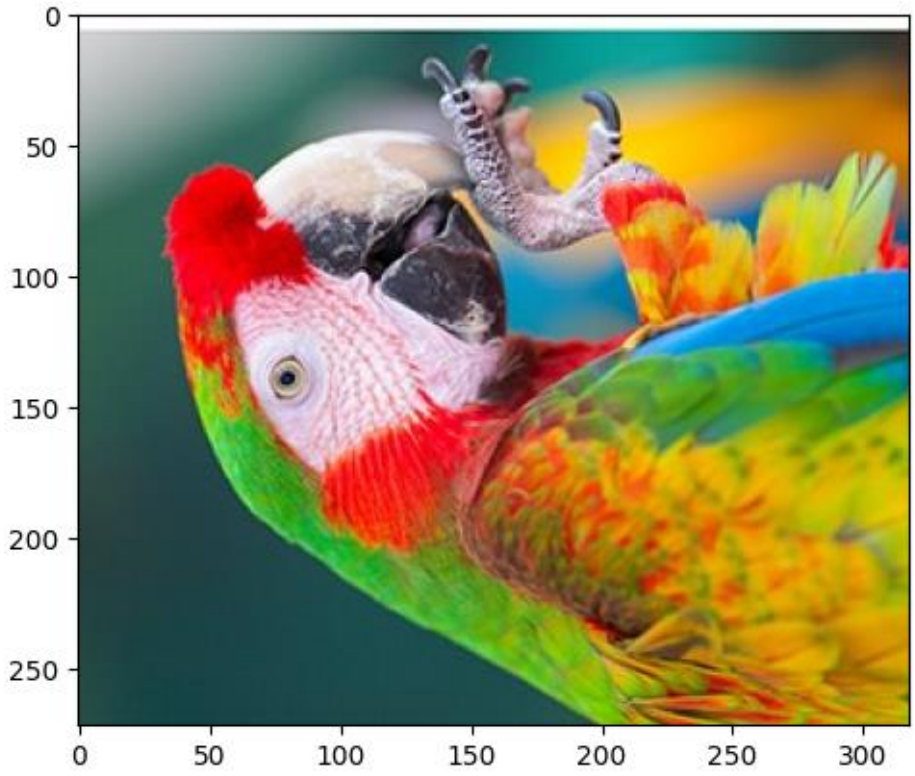
```
<matplotlib.image.AxesImage at 0x1c570e51310>
```




```
uptodown_image = np.flipud(image)

plt.imshow(uptodown_image)

<matplotlib.image.AxesImage at 0x1c571ec4550>
```

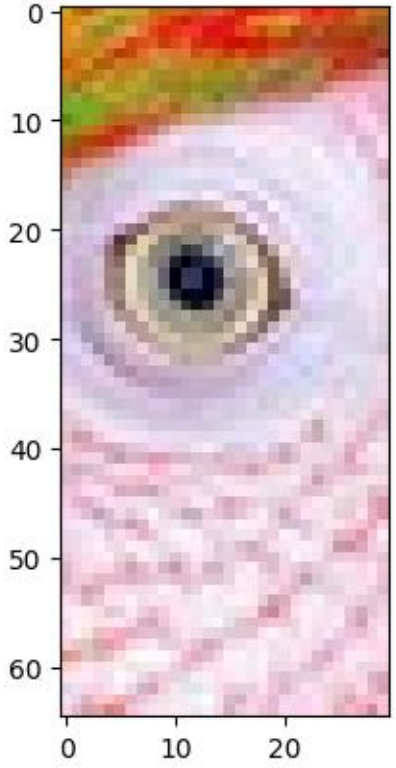



```
rotate_image = np.rot90(image)

plt.imshow(rotate_image)

<matplotlib.image.AxesImage at 0x1c571f22ad0>
```



plt.imshow(image[55:120,120:150])

<matplotlib.image.AxesImage at 0x1ff548c2210>



Joining Arrays

Joining Arrays

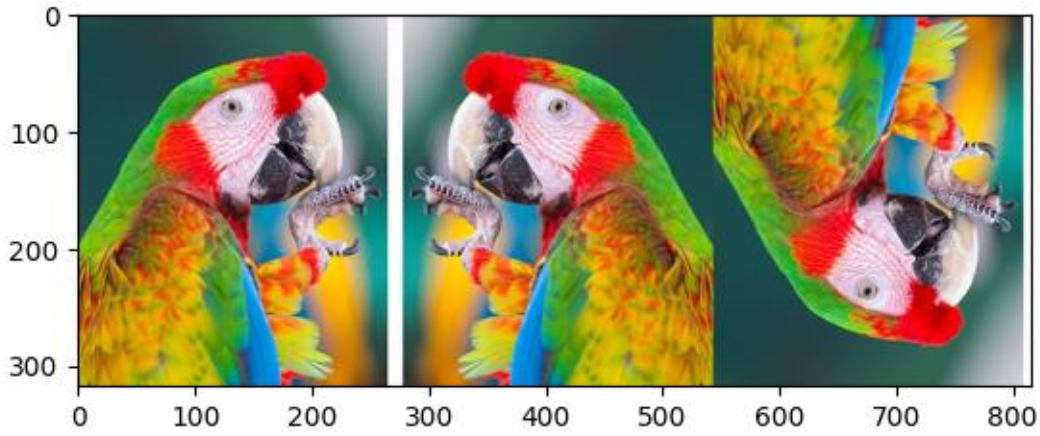
We can combine arrays either **horizontally** or **vertically**.

Functions:

- **np.concatenate((a, b), axis=0/1)** → Concatenates arrays.
- **np.vstack((a, b))** → Vertical stack (row-wise).
- **np.hstack((a, b))** → Horizontal stack (column-wise).
- **np.dstack((a, b))** → Stack along depth (3D).
- **np.stack((a, b), axis)** → Stack arrays along new axis.

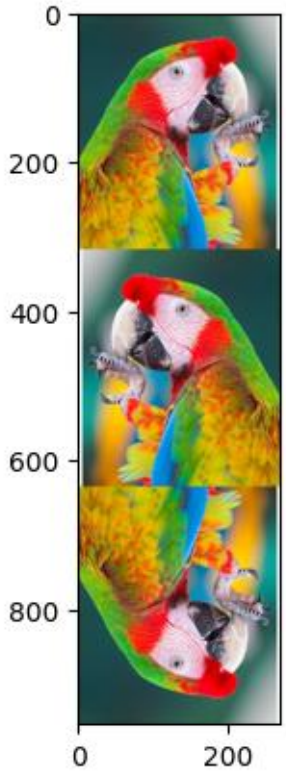
```
plt.imshow(np.hstack((image,flipped_image,uptodown_image)))
```

```
<matplotlib.image.AxesImage at 0x1ff549439d0>
```



plt.imshow(np.vstack((image,flipped_image,uptodown_image)))

<matplotlib.image.AxesImage at 0x1ff54975450>



Searching and Sorting in NumPy

NumPy provides efficient functions to **search, filter, and sort** data in arrays. These operations are much faster than Python's built-in methods because NumPy works on **C-optimized arrays**.

1. Searching in Arrays

Searching means **finding positions, conditions, or values** inside an array.

(a) np.where() – Conditional Search

Returns indices where the condition is true.

Example:

```
import numpy as np


arr = np.array([10, 20, 30, 40, 50])
result = np.where(arr > 25)
print("Indices where arr > 25:", result)
print("Values:", arr[result])
```

Output:

```
Indices: (array([2, 3, 4]),)
Values: [30 40 50]
```

(b) np.nonzero() – Non-zero Elements

Finds indices of non-zero elements.

Example:

```
arr = np.array([0, 5, 0, 7, 9])
print("Non-zero indices:", np.nonzero(arr))
```

Output:

```
(array([1, 3, 4]),)
```

(c) np.argmax() and np.argmin()

Finds index of maximum and minimum element.

Example:

```
arr = np.array([12, 5, 18, 7, 3])
print("Index of max:", np.argmax(arr))
print("Index of min:", np.argmin(arr))
```

Output:

```
Index of max: 2
Index of min: 4
```

(d) np.searchsorted() – Binary Search

Finds index where a value should be inserted to maintain sorted order.

Example:

```
arr = np.array([10, 20, 30, 40])
```

```
print("Insert position for 25:", np.searchsorted(arr, 25))
print("Insert position for 5:", np.searchsorted(arr, 5))
```

Output:

```
Insert position for 25: 2
Insert position for 5: 0
```

Sorting in Arrays

Sorting means arranging elements in ascending or descending order.

(a) np.sort() – Returns Sorted Array

Example:

```
arr = np.array([7, 2, 9, 4, 1])
print("Sorted:", np.sort(arr))
```

Output:

```
[1 2 4 7 9]
```

(b) np.argsort() – Indices of Sorted Order

Instead of sorting values, it returns the **indices** that would sort the array.

Example:

```
arr = np.array([50, 10, 40, 20])
print("Argsort:", np.argsort(arr))
print("Sorted using indices:", arr[np.argsort(arr)])
```

Output:

```
[1 3 2 0]
```

[10 20 40 50]

(c) Sorting 2D Arrays

You can sort along rows or columns.

Example:

```
mat = np.array([[3, 1, 2],
                [9, 7, 8]])


print("Sort along axis=0 (columns):\n", np.sort(mat, axis=0))
print("Sort along axis=1 (rows):\n", np.sort(mat, axis=1))
```

(d) Descending Sort

Use slicing ([::-1]) after sort.

Example:

```
arr = np.array([15, 3, 9, 27])
print("Descending:", np.sort(arr)[::-1])
```

(e) np.partition() – Partial Sorting

Places the k-th smallest element in its correct position and rearranges others (faster than full sort).

Example:

```
arr = np.array([7, 2, 9, 4, 1])
print("Partition (k=2):", np.partition(arr, 2))
```

Output:

[1 2 4 9 7] # 3rd smallest (4) is in correct place

Combining Searching and Sorting

- **Find largest/smallest values quickly** using np.argpartition().
- **Filter values** using np.where() and then sort them.

Example:

```
arr = np.array([12, 7, 19, 5, 3, 15])

# Find top 3 largest values

indices = np.argpartition(arr, -3)[-3:]

print("Top 3 values:", arr[indices])

print("Sorted Top 3:", np.sort(arr[indices]))
```

NaN Statistics in NumPy

NaN (**Not a Number**) represents missing or undefined values. NumPy provides special functions to **handle NaN values** because normal statistical functions like np.mean() will return NaN if even one NaN exists.

Common NaN-safe Functions:

- **np.isnan(x)** → Checks if values are NaN.
- **np.nanmean(x)** → Mean ignoring NaN.
- **np.nanstd(x)** → Standard deviation ignoring NaN.
- **np.nanvar(x)** → Variance ignoring NaN.
- **np.nanmin(x), np.nanmax(x)** → Min & Max ignoring NaN.
- **np.nansum(x)** → Sum ignoring NaN.

Example:

```
import numpy as np


arr = np.array([10, np.nan, 20, 30, np.nan])


print("Is NaN:", np.isnan(arr))
print("Nan Mean:", np.nanmean(arr))  # (10+20+30)/3 = 20
print("Nan Sum:", np.nansum(arr))     # 60
print("Nan Min:", np.nanmin(arr))     # 10
print("Nan Max:", np.nanmax(arr))     # 30
```

NumPy Iterators (np.nditer())

np.nditer() allows you to **iterate over elements of arrays efficiently**, regardless of shape or dimension.

(a) Basic Iteration

```
arr = np.array([[1, 2], [3, 4]])

for x in np.nditer(arr):
    print(x, end=" ")
```

Output:

1 2 3 4

(b) Iterating with Indices (np.ndenumerate())

```
for idx, val in np.ndenumerate(arr):
    print("Index:", idx, "Value:", val)
```

Output:

Index: (0, 0) Value: 1

Index: (1, 0) Value: 3

Index: (1, 1) Value: 4

(c) Iterating with Operations

```
for x in np.nditer(arr, flags=['external_loop'], order='C'):
    print("Chunk:", x)
```

Useful when handling **multi-dimensional arrays**.

---

np.where()

Used to **search** in arrays.

(a) Condition-based Selection

```
arr = np.array([5, 12, 7, 18, 3])
result = np.where(arr > 10)
print("Indices where arr > 10:", result)
print("Values:", arr[result])
```

Output:

```
Indices: (array([1, 3]),)
Values: [12 18]
```

(b) Replace Values Based on Condition

```
arr = np.array([5, 12, 7, 18, 3])
new_arr = np.where(arr > 10, 1, 0)  # if >10 → 1 else 0
print(new_arr)
```

Output:

```
[0 1 0 1 0]
```

---

4. np.clip()

np.clip() is used to **limit values** of an array within a given range.

Syntax:

```
np.clip(arr, min_value, max_value)
```

Example:

```
arr = np.array([2, 8, 15, 4, 20])
clipped = np.clip(arr, 5, 15)
```

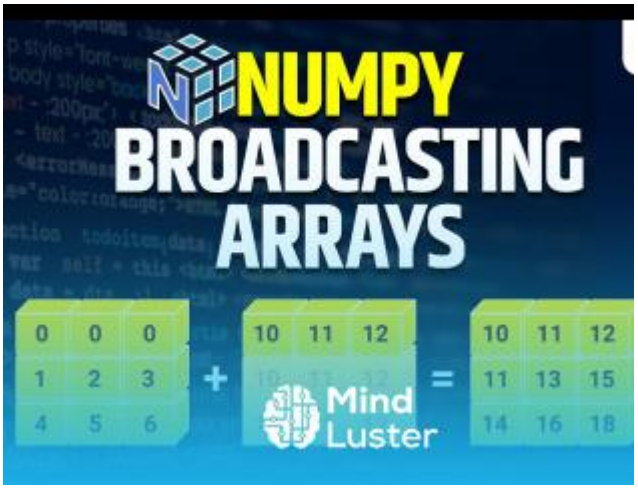
```
print("Original:", arr)

print("Clipped:", clipped)
```

Output:

```
Original: [ 2  8 15  4 20]

Clipped: [ 5  8 15  5 15]
```

Values **below 5** become **5**, and values **above 15** become **15**.

Array Broadcasting in NumPy



Broadcasting is a powerful feature in NumPy that allows arrays of **different shapes** to be used together in arithmetic operations **without explicitly copying data**.

Instead of looping over elements manually, NumPy automatically **“stretches” smaller arrays** to match the shape of larger arrays.

Why Broadcasting?

Normally, to perform operations, arrays must have the **same shape**. But with broadcasting, NumPy makes them compatible.

Example without broadcasting (same shape required):

```
import numpy as np

a = np.array([1, 2, 3])

b = np.array([4, 5, 6])

print(a + b)   # [5 7 9]
```

But broadcasting allows:

```
a = np.array([1, 2, 3])

b = 5

print(a + b)   # [6 7 8]
```

Here, scalar 5 is **broadcast** to [5, 5, 5].

Rules of Broadcasting

When operating on two arrays, NumPy compares their **shapes** element by element from **right to left**.

- If dimensions are equal → ✅ Compatible
- If one dimension is 1 → ✅ It will be **stretched**
- Otherwise → ❌ Incompatible

Example Shapes:

(3, 1) and (1, 4) → broadcast to (3, 4)

(5, 4) and (1, 4) → broadcast to (5, 4)

(2, 3, 4) and (3, 1) → broadcast to (2, 3, 4)

---

3. Examples of Broadcasting

(a) Scalar with Array

```
arr = np.array([1, 2, 3, 4])
print(arr * 10)   # [10 20 30 40]
```

Scalar 10 is broadcast across all elements.

(b) 1D with 2D Array

```
mat = np.array([[1, 2, 3],
                [4, 5, 6]])

vec = np.array([10, 20, 30])
print(mat + vec)
```

Output:

```
[[11 22 33]
 [14 25 36]]
```

Here, [10, 20, 30] was broadcast to both rows.

(c) Column Vector with Row Vector

```
a = np.array([[1], [2], [3]])   # Shape (3,1)
b = np.array([10, 20, 30])      # Shape (3,)
```

```
print(a + b)
```

Output:

```
[[11 21 31]
 [12 22 32]
 [13 23 33]]
```

Common Use Cases

(a) Normalization (subtract mean, divide by std)

```
X = np.array([[1, 2, 3],
              [4, 5, 6],
              [7, 8, 9]])


mean = X.mean(axis=0)   # column means [4 5 6]
print("Normalized:\n", X - mean)
```

(b) Applying Constants

```
image = np.array([[100, 150, 200],
                  [50,  75,  125]])


brightness = 20
print(image + brightness)
```

(c) Mathematical Operations on Grids

```
x = np.arange(3).reshape(3,1)
y = np.arange(3).reshape(1,3)
print("X:\n", x)
print("Y:\n", y)
print("X+Y:\n", x + y)
```

Output:

```
X:
[[0]
 [1]
 [2]]

Y:
[[0 1 2]]

X+Y:
[[0 1 2]
 [1 2 3]
 [2 3 4]]
```

Advantages of Broadcasting

Saves memory → No need to copy arrays.
Faster computation → Uses vectorized operations.
Clean code → No need for nested loops.

6. Broadcasting Errors

Not all shapes are compatible.

```
a = np.array([1, 2, 3])    # Shape (3,)
b = np.array([1, 2])       # Shape (2,)
print(a + b)   # ❌ Error (shapes mismatch)
```

Set Operations in NumPy

NumPy provides functions to perform **mathematical set operations** on arrays, similar to Python sets but working **element-wise** on arrays.

(a) Unique Elements

- **np.unique(arr)** → Returns sorted unique values.

Example:

```
import numpy as np
arr = np.array([1, 2, 2, 3, 4, 3])
print("Unique elements:", np.unique(arr))
```

Output:

```
[1 2 3 4]
```

(b) Union, Intersection, Difference

- **np.union1d(a, b)** → Union of two arrays.
- **np.intersect1d(a, b)** → Intersection.

- **np.setdiff1d(a, b)** → Difference (a - b).
- **np.setxor1d(a, b)** → Symmetric difference.

Example:

```
a = np.array([1, 2, 3])

b = np.array([2, 3, 4])


print("Union:", np.union1d(a, b))

print("Intersection:", np.intersect1d(a, b))

print("Difference (a-b):", np.setdiff1d(a, b))

print("Symmetric difference:", np.setxor1d(a, b))
```

Linear Algebra in NumPy

NumPy has a dedicated **np.linalg module** for linear algebra operations.

(a) Dot Product

- **np.dot(a, b)** → Matrix multiplication or inner product.

Example:

```
a = np.array([1, 2])

b = np.array([3, 4])

print("Dot product:", np.dot(a, b))  # 1*3 + 2*4 = 11
```

(b) Matrix Multiplication (2D)

```
A = np.array([[1, 2], [3, 4]])

B = np.array([[5, 6], [7, 8]])
```

```
print("Matrix multiplication:\n", np.dot(A, B))
```

(c) Determinant

- **np.linalg.det(A)** → Computes determinant.

Example:

```
A = np.array([[1, 2], [3, 4]])
print("Determinant:", np.linalg.det(A))
```

(d) Inverse of a Matrix

- **np.linalg.inv(A)** → Inverse (if determinant ≠ 0)

Example:

```
A = np.array([[1, 2], [3, 4]])
print("Inverse:\n", np.linalg.inv(A))
```

(e) Eigenvalues and Eigenvectors

- **np.linalg.eig(A)** → Returns eigenvalues and eigenvectors.

Example:

```
A = np.array([[2, 0], [0, 3]])
eigenvalues, eigenvectors = np.linalg.eig(A)
print("Eigenvalues:", eigenvalues)
print("Eigenvectors:\n", eigenvectors)
```

(f) Norm of a Vector or Matrix

- **np.linalg.norm(x)** → Computes Euclidean norm (length) of vector.

Example:

```
v = np.array([3, 4])

print("Vector norm:", np.linalg.norm(v))  # sqrt(3^2 + 4^2) = 5
```

---

3. Matrix Operations in NumPy

Matrix operations are fundamental for linear algebra and data analysis.

(a) Transpose

```
A = np.array([[1, 2], [3, 4]])

print("Transpose:\n", A.T)
```

(b) Trace

- **np.trace(A)** → Sum of diagonal elements.

```
print("Trace:", np.trace(A))  # 1 + 4 = 5
```

(c) Matrix Rank

- **np.linalg.matrix_rank(A)** → Returns rank of matrix.

```
print("Rank:", np.linalg.matrix_rank(A))
```

(d) Solve Linear System

Solve AX = B → X = np.linalg.solve(A, B)

Example:

```
A = np.array([[3, 1], [1, 2]])

B = np.array([9, 8])
```

```
X = np.linalg.solve(A, B)

print("Solution:", X)  # X satisfies AX = B
```

(e) Matrix Power

- **np.linalg.matrix_power(A, n)** → A^n

Example:

```
A = np.array([[2, 0], [0, 3]])

print("A squared:\n", np.linalg.matrix_power(A, 2))
```

File Handling

NumPy provides efficient functions to **save and load arrays** in **text or binary formats**, which is much faster than using Python's standard file I/O for numerical data.

Example:

```
salary = np.random.randint(15000,60000,20)
```

Output:

```
array([57996, 25086, 54455, 44039, 28188, 57563, 29797, 20889, 20408,
       51642, 33905, 20982, 31241, 32035, 17456, 20717, 48830, 30071,
       47886, 29009], dtype=int32)


updated_salary = np.where(salary<25000,salary +0.1*salary,salary + 0.05*salary)
```

```
np.savetxt(r"C:\Users\ghema\Latest Salary.txt"'Latest Salary.txt',updated_salary)

updated_salary
```

Output:

```
array([60895.8 , 26340.3 , 57177.75, 46240.95, 29597.4 , 60441.15,
       31286.85, 22977.9 , 22448.8 , 54224.1 , 35600.25, 23080.2 ,
       32803.05, 33636.75, 19201.6 , 22788.7 , 51271.5 , 31574.55,
       50280.3 , 30459.45])
```

```
np.loadtxt(r"C:\Users\ghema\Latest Salary.txt")
```

Output:

```
array([60895.8 , 26340.3 , 57177.75, 46240.95, 29597.4 , 60441.15,
       31286.85, 22977.9 , 22448.8 , 54224.1 , 35600.25, 23080.2 ,
       32803.05, 33636.75, 19201.6 , 22788.7 , 51271.5 , 31574.55,
       50280.3 , 30459.45])
```

```
np.save(r"C:\Users\ghema\Latest Salary.txt"'Latest Salary.npy',updated_salary)
np.load(r"C:\Users\ghema\Latest Salary.txtLatest Salary.npy")
```

Output:

```
array([60895.8 , 26340.3 , 57177.75, 46240.95, 29597.4 , 60441.15,
       31286.85, 22977.9 , 22448.8 , 54224.1 , 35600.25, 23080.2 ,
       32803.05, 33636.75, 19201.6 , 22788.7 , 51271.5 , 31574.55,
       50280.3 , 30459.45])
```

Splitting Arrays

```
import numpy as np
a = np.array([[1,2,3],[4,5,6],[7,8,9]])
a
```

Output:

```
array([[1, 2, 3],
   [4,5,6],
[[7, 8, 9]])
np.hsplit(a,3)   # Column wise splitting
```

Output:

```
[array([[1],
        [4],
        [7]]),
 array([[2],
        [5],
        [8]]),
 array([[3],
        [6],
        [9]])]

np.vsplit(a,3)  # row wise  splitting
```

[array([[1, 2, 3]]), array([[4, 5, 6]]), array([[7, 8, 9]])]

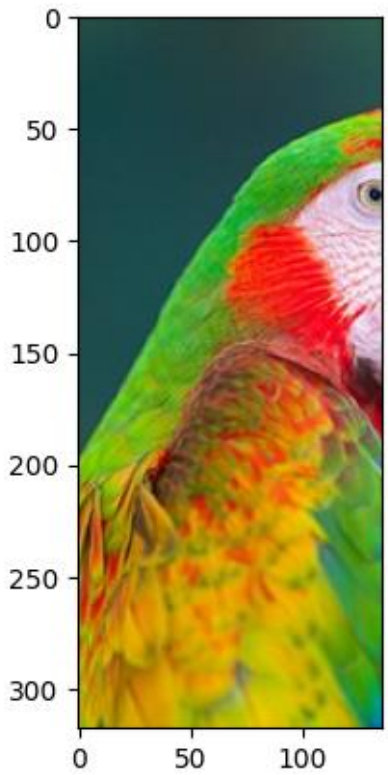
```
import matplotlib.pyplot as plt
image = plt.imread(r"C:\Users\ghema\Downloads\image.jpg")
image.shape
```

Output:

(318, 272, 3)

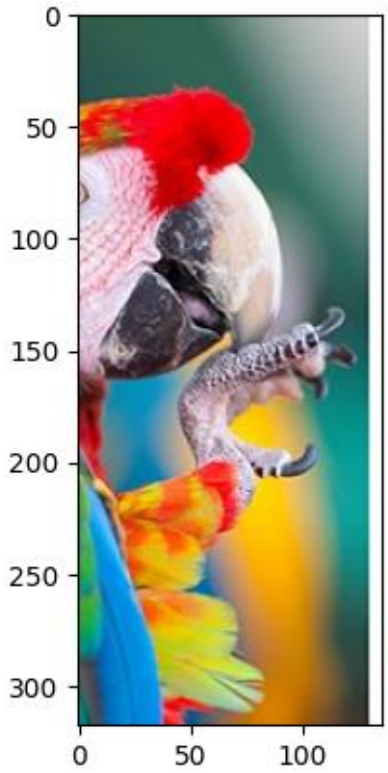
```
x,y = np.hsplit(image,2)
plt.imshow(x)
```

<matplotlib.image.AxesImage at 0x1dcf8a2b770>



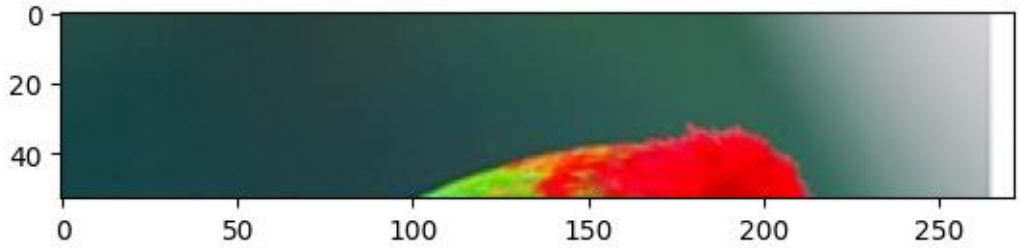

```
plt.imshow(y)
```

<matplotlib.image.AxesImage at 0x1dcf8aff750>



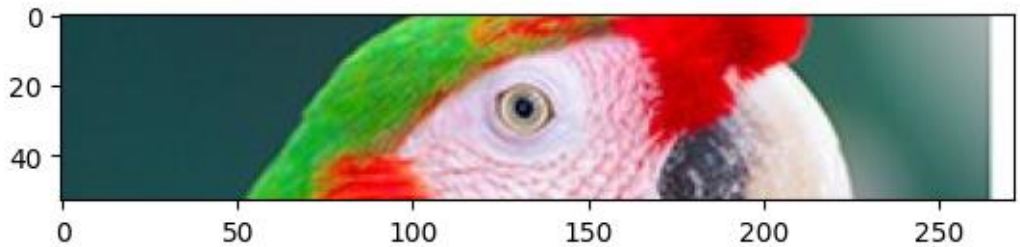
a,b,c,d,e,f = np.vsplit(image,6)

<matplotlib.image.AxesImage at 0x1dcfada9810>



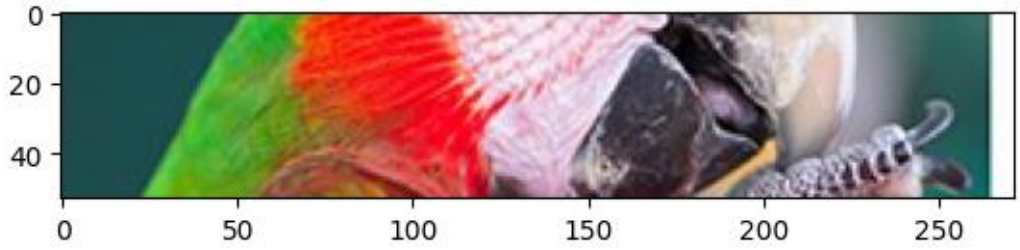
plt.imshow(b)

<matplotlib.image.AxesImage at 0x1dcfacd1f90>



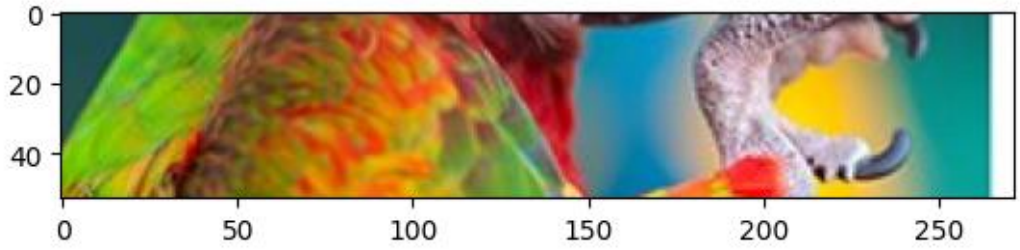
plt.imshow(c)

<matplotlib.image.AxesImage at 0x1dcfadf2710>



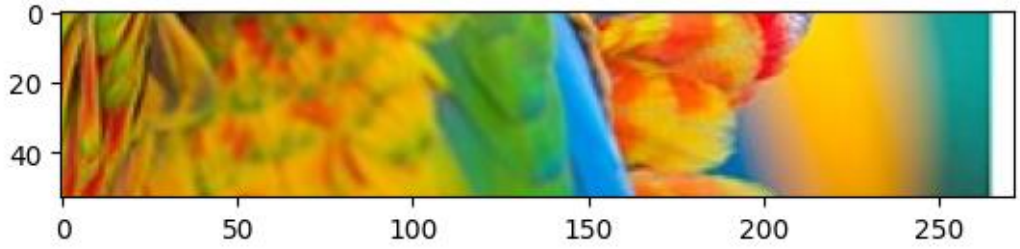
plt.imshow(d)

<matplotlib.image.AxesImage at 0x1dcfae2ae90>



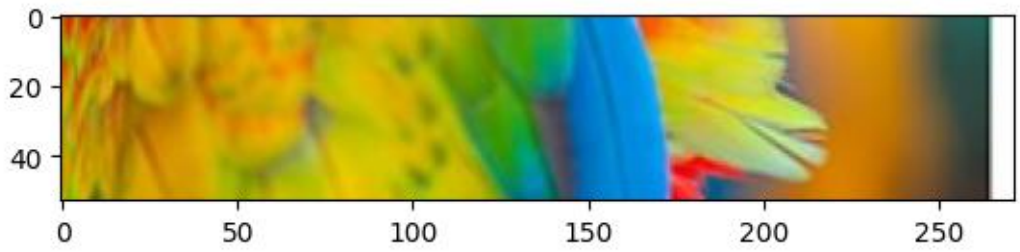
plt.imshow(e)

<matplotlib.image.AxesImage at 0x1dcfae63890>



plt.imshow(f)

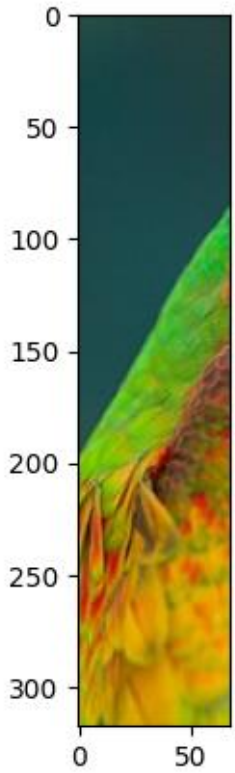
<matplotlib.image.AxesImage at 0x1dcfaec8190>




```
x,y,t,z = np.hsplit(image,4)
```

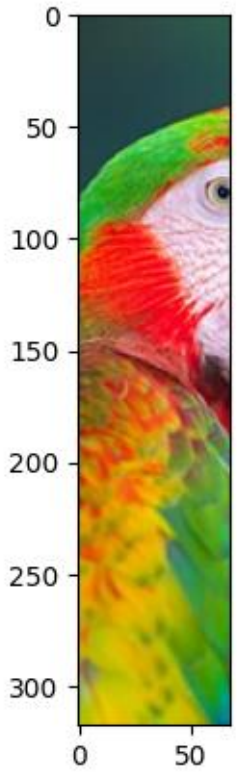
```
plt.imshow(x)
```

```
<matplotlib.image.AxesImage at 0x1dcfb0b1810>
```




```
plt.imshow(y)
```

```
<matplotlib.image.AxesImage at 0x1dcfb10df90>
```



Distributions

Distributions describe how values are spread. NumPy provides functions to **generate random numbers** following different **probability distributions**, which is useful in **simulation, statistics, and machine learning**.

NumPy uses the **numpy.random module** for distributions.

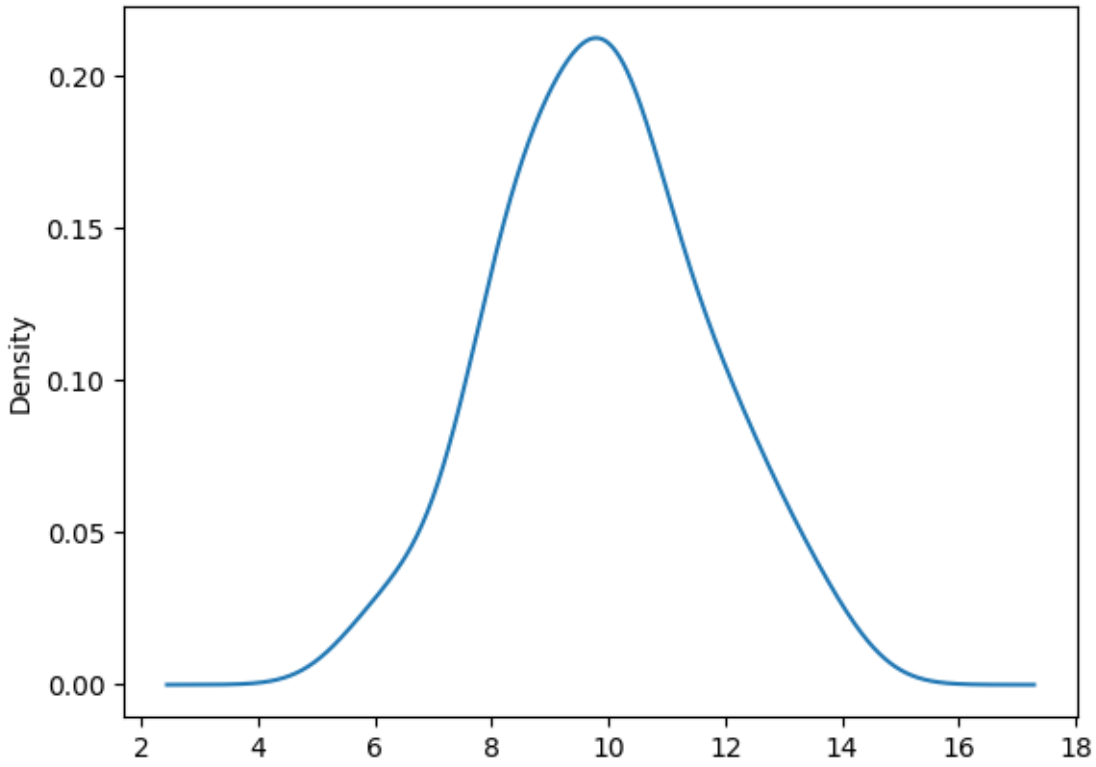
```
!pip install pandas

import numpy as np

np.random.seed(435)

norm = np.random.normal(loc = 10,scale = 2,size = 40)

import pandas as pd

pd.Series(norm).skew()
```

Output:

np.float64(0.09146948716911671)

pd.Series(norm).plot(kind = 'kde')

<Axes: ylabel='Density'>



Conclusion

NumPy is one of the most important Python libraries for numerical and scientific computing. It provides fast and efficient handling of large arrays and matrices, which makes it better than normal Python lists. With NumPy, we can easily perform mathematical, logical, and statistical operations in just a few lines of code.

It supports a wide range of features such as array creation, reshaping, slicing, indexing, broadcasting, and file handling. NumPy also includes powerful tools for linear algebra, random number generation, probability distributions, and data manipulation. These features make it the backbone of data science, machine learning, artificial intelligence, and many research fields.

In short, NumPy saves time, reduces memory usage, and allows us to work with data more effectively. Without NumPy, most data analysis and machine learning tasks in Python would be slow and difficult. That is why NumPy is considered the foundation for almost all advanced Python libraries like Pandas, SciPy, TensorFlow, and many more